

BufferedWriter

Efficient Character Writing Strategies

codehive

codehive

Advanced Text Writing

While `FileWriter` is useful for simple tasks, `BufferedWriter` is the standard for professional applications. It reduces the number of physical write operations by buffering data in memory before committing it to the disk.

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class WriteFile {
    public static void main(String[] args) {
        try (BufferedWriter bw = new BufferedWriter(new
FileWriter("filename.txt"))) {
            bw.write("Hello World!");
            bw.newLine(); // Platform-independent line break
            bw.write("BufferedWriter is fast!");
        } catch (IOException e) {
            System.out.println("Error writing file.");
        }
    }
}
```

Why use BufferedWriter?

1. The `newLine()` Method: Unlike manually adding `\n`, the `newLine()` method uses the system's own line separator property (e.g., CRLF on Windows vs. LF on Unix), ensuring your file is readable across all platforms.

2. Performance Gain: Writing to disk is a "heavy" operation. `BufferedWriter` collects data into an internal buffer (default 8KB) and writes it in a single "burst," making it significantly faster than `FileWriter` alone.

3. Resource Protection: When used with Try-with-Resources, the buffer is automatically "flushed" (cleared into the file) before the stream is closed, preventing trailing data loss.

Optimized Character Storage with

codehive

Code faster. Write smarter.